

2026-07-30 更新:修复 gate v3 训练/推理不一致 + BGE 检索指令前缀

一、背景

多跳问答系统每一跳检索候选证据后，用 `gate v3(h_after ⊕ Δ` 特征融合，见 0727 更新)对候选打分，跟余弦相似度加权混合排序：

```
final_score = emb_score - λ · gate_v3_score
```

MuSiQue 的子问题里，后续跳经常需要引用前面跳的答案，写法是 `[Answer N]` 占位符（例如 "When was [Answer 1] founded?"）。0727 更新第 5 节已经发现：**gate v3 训练时读到的子问题占位符 是没有展开的**（训练数据 `hidden_states/pilot_multilayer` 来自 `traces/merged/musique/*.jsonl` 的 `reasoning_trace` 字段，这个字段从未调用过 `expand_hop_template`），但推理时（`run_retrieval_exp_wavefront_gate_v3.py`）子问题会先用前面跳生成的短答案展开占位符，再用来检索、也用来喂给 `gate` 打分——这是一个训练/推理不一致，影响约 49% 的跳。

0727 更新第 5.4 节尝试过一版"检索和 `gate` 打分都不展开占位符"（`_decoupled.py`），结果是负面的：四项指标全面下降（`recall@1` -5.93、`chain` -9.80 个百分点），按跳位置拆分确认问题出在**检索**这一步——原始子问题残缺，余弦检索（BGE，通用语义模型，没见过这条链路）找不到正确候选。第一跳（不需要占位符）两版完全一致，从第二跳开始不展开版全面更差，`K` 越大掉得越狠。

这次更新做的是一个更精确的修复：**只修 `gate` 打分这一处，完全不碰检索**。

二、方法一：rawprefix——给 gate 打分单独维护一条"原始版"累计前缀

新建 `retrieval/run_retrieval_exp_wavefront_gate_v3_rawprefix.py`（替换了之前效果为负的 `_decoupled.py`），每条候选路径维护两条并行的累计记录，而不是一条：

- hop_steps（展开版）**：跟现有 v3 完全一样，（**展开后的子问题，证据文本**），用于检索查询展开、逐跳短答案生成的上下文、最终 reader 的上下文——**这三处全部不变**
- gate_hop_steps（原始版，新增）**：（**原始子问题，证据文本**），只用来拼喂给 `gate v3` 打分的累计前缀——从第一跳到当前跳，`gate` 看到的整条前缀里子问题全部是原始未展开的，跟训练时的分布完全一致（不只是当前这一跳，是整条前缀）

检索、逐跳短答案生成、最终 reader 三处的代码逻辑跟现有 `run_retrieval_exp_wavefront_gate_v3.py` 完全相同，唯一的差别是 `gate` 打分喂的文本换了一份。

musique dev 全量结果（ $\lambda=0.50$ ，即 0727 更新第六节调好的默认值）

	recall@1	chain@1	EM	F1
现有 v3（ $\lambda=0.50$ ，展开占位符）	0.6984	0.5114	0.4249	0.5187
rawprefix（gate 打分用原始版）	0.7008	0.5143	0.4274	0.5216

	recall@1	chain@1	EM	F1
差值	+0.24	+0.29	+0.25	+0.29

四项指标全部小幅提升，检索侧没有像"完全不展开"那版一样下降，反而也涨了一点——确认"只改 gate 打分文本、不动检索"这个更精确的修复方式是有效的，之前那版负面结果是因为检索和打分两个用途没有 分开处理。

三、方法二：BGE 检索指令前缀

顺带查了一下检索这一步的实现（`retrieval/run_retrieval_exp.py::embed_retrieval`），发现查询文本 是直接编码的，没有加 **BGE 官方文档推荐的检索指令前缀**：

```
qv = st.encode([query], normalize_embeddings=True) # 原文直接编码，没有指令前缀
```

bge-base-en-v1.5 官方推荐：非对称检索场景（短 query 找长 passage）下，query 编码前应该加一句 **"Represent this sentence for searching relevant passages: "**，帮助模型区分"这是查询"和"这是普通 文本"。这是全项目公用的函数（baseline、gate_v2、gate_v3、rawprefix 全部复用），没有单独针对 某个方法缺失，是从一开始就没加。

不能直接改 `embed_retrieval()`（生产共用函数，其它方法都依赖它），在 `rawprefix` 脚本里新增 `--query-instruction` 参数，只对传进 `embed_retrieval` 的查询文本加前缀，默认就是 **BGE 官方推荐** 的那句，传空字符串可以还原成不加前缀的旧行为方便对比。

musique dev 全量结果（在 `rawprefix` 基础上再加这个前缀）

	recall@1	chain@1	EM	F1
rawprefix（无检索前缀）	0.7008	0.5143	0.4274	0.5216
rawprefix + BGE 检索前缀	0.7018	0.5143	0.4299	0.5256
差值	+0.10	持平	+0.25	+0.40

同样是全指标不降、大部分小幅提升。

四、两个改动叠加的总效果（相对最初的 v3 $\lambda=0.50$ ）

	recall@1	chain@1	EM	F1
v3 ($\lambda=0.50$ ，原始)	0.6984	0.5114	0.4249	0.5187
+ rawprefix	0.7008	0.5143	0.4274	0.5216
+ BGE 检索前缀	0.7018	0.5143	0.4299	0.5256
累计差值	+0.34	+0.29	+0.50	+0.69

两个改动方向都对，而且可以叠加，没有互相抵消。涨幅比 0727 更新第六节的 λ 调参（4-11 个百分点）小一个量级，但方向一致、四项都没有下降。

五、还没做完的部分

- **2WikiMultihopQA、HotpotQA 上的验证还没跑**——目前 rawprefix + BGE 前缀这套结果只在 musique 上跑过，跟 0727 更新第七节验证 $\lambda=0.50$ 泛化性时一样，理论上也需要在另外两个数据集上确认这次的提升不是只对 musique 有效，这一步因为显存资源紧张还没来得及跑
- **真正的 recall@3 指标**：之前发现 `--beam-width 1` 时脚本自带的 `oracle_gold_rank_among_survivors` 字段里 recall@1/recall@3/mrr 会退化成同一个数（因为剪枝后只剩 1 个候选，没有第二三名可比）。已经在 rawprefix 脚本里加了 `full_pool_gold_rank_overall/full_pool_gold_rank_by_K`，在剪枝到 beam_width 之前、对完整的 `--retrieve-k` 候选池算真正的 recall@1/recall@3——这个改动加上去之后还没有实际跑出结果，是下次要补的数据。